

dlegoream system

[README.MD](#)

DLEGOREAM LABORATORY



문제인식:

해결:



로드맵



Dlegoream System

플러그인 기반 검색 / 크롤링 / 다운로드 파이프라인 시스템입니다.

로컬 파일, 외부 데이터 소스(arXiv, Wikipedia 등), 다운로드 대상 데이터를 하나의 검색 엔진 구조로 통합하는 것을 목표로 합니다.

주요 기능

플러그인 기반 검색 구조

검색 결과 스코어링 시스템

확장 가능한 크롤러 구조

인버티드 인덱싱 지원

다운로드 품질 플러그인 지원

메타데이터 기반 추천 및 정렬

SQLite 기반 저장소 연동 가능

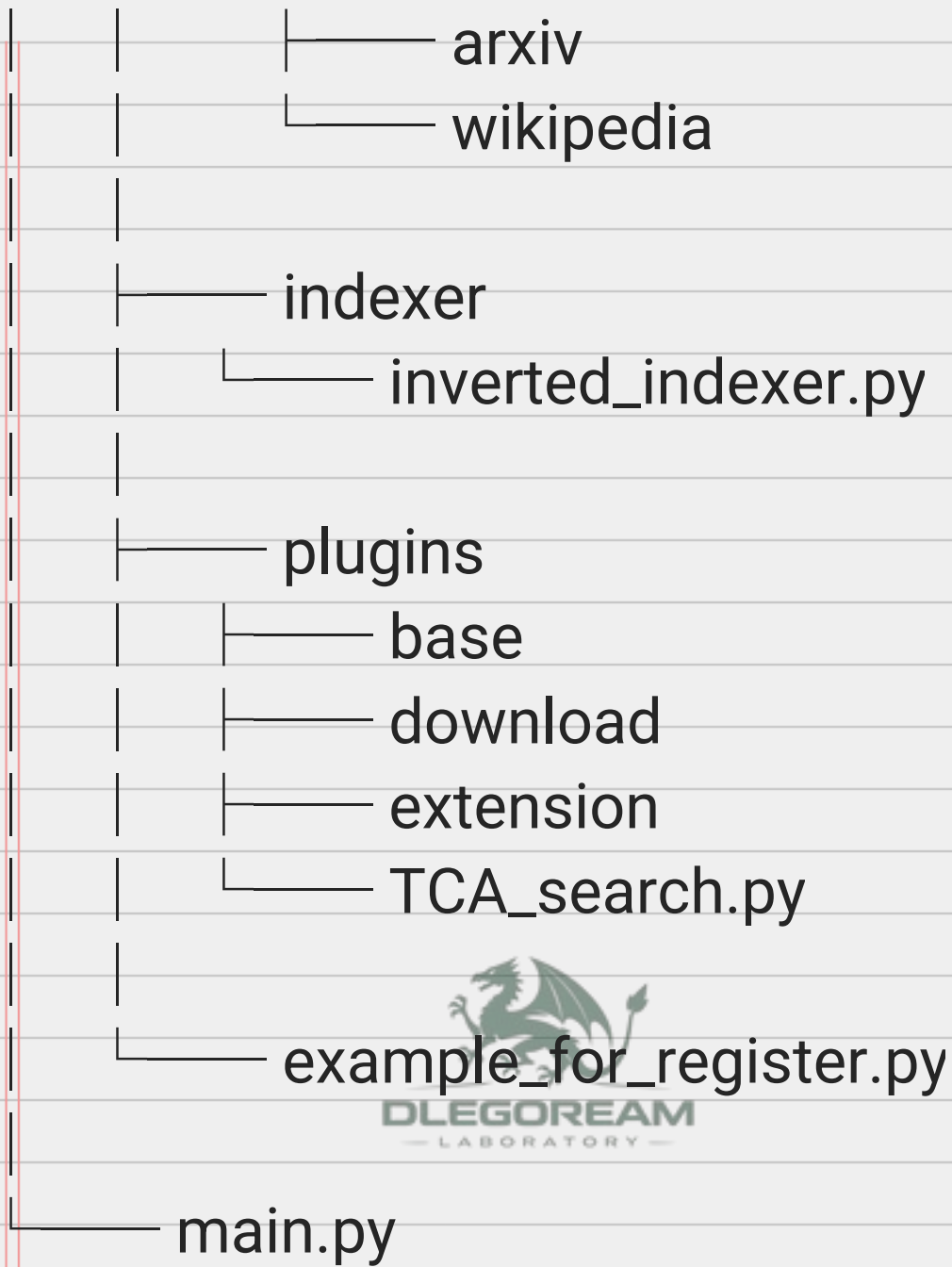


프로젝트 구조

bash



```
.
├── dlegoream_system
│   ├── config
│   │   ├── config.py
│   │   ├── schema.py
│   │   └── weight_config.json
│   ├── core
│   │   ├── engine.py
│   │   ├── model.py
│   │   ├── indexer.py
│   │   └── interface
│   │       ├── crawler.py
│   │       ├── plugin.py
│   │       ├── retrieve.py
│   │       ├── scorer.py
│   │       └── storage.py
│   └── crawler
│       └── source
```



핵심 아키텍처

1. Search Engine

SearchEngine은 전체 검색 흐름을 담당합니다.

동작 순서:

1. 저장소(Storage)에서 데이터 로드
2. Plugin Filter 적용
3. Plugin Scoring 적용
4. Schema 기반 추가 점수 계산
5. 추천 데이터 처리
6. 결과 정렬 후 반환

```
python
```



```
engine = SearchEngine(storage,  
plugin_manager)  
results = engine.search(query, path)
```



DLEGOREAM
— LABORATORY —

2. Plugin System

플러그인을 이용하여 기능을 자유롭게 확장할 수 있습니다.

예시:

검색 스코어링

품질 선택

필터링

다운로드 옵션

결과 추천

예시 등록

python



```
quality_manager.register("1080",  
P1080())  
score_manager.register(SimpleScorePl  
ugin())
```

















3. Crawler

외부 데이터 소스를 탐색합니다.

현재 포함된 예시:

arXiv 검색

Wikipedia 검색

예시:

```
python
```



```
records = crawler.explore(query.text)
```

4. Indexer

검색 속도를 높이기 위한 인버티드 인덱스를 지원합니다.

```
python
```



```
from  
dlegoream_system.indexer.inverted_ind  
exer import InvertedIndexer
```

실행 예제

기본 실행

```
bash
```



```
python main.py
```

python



```
from
dlegoream_system.storage.storage_ba
ckends import SQLiteStorage
from
dlegoream_system.service.record_adap
ter import RecordAdapter
from dlegoream_system.core.engine
import SearchEngine
from
dlegoream_system.plugins.manager
import PluginManager
```

```
storage = SQLiteStorage()
adapter = RecordAdapter(storage)
pm = PluginManager()
```

```
engine = SearchEngine(storage, pm)
```

데이터 저장 예시

python



```
from dlegoream_system.core.model
import FileRecord
import time
```

```
record = FileRecord(
    id="test-1",
    path="/storage/project/main.py",
    created_at=time.time(),
    updated_at=time.time(),
    last_seen=time.time(),
    type="local",
    meta={"content": "hello world"}
)
```

```
adapter.save(record)
```

검색 예시

python



```
results = engine.search(  
    query="test",  
    path="/storage/project"  
)  
  
for r in results:  
    print(r.record.path, r.score)
```

파이프라인 예제

example_for_register.py 파일에는 다음 흐름이 구현되어 있습니다.

1. 크롤링
2. 점수 계산
3. 필터링
4. 다운로드 포맷 선택
5. 다운로드 실행

python



```
asyncio.run(  
    run_pipeline("ai research paper")  
)
```

확장 아이디어

- ElasticSearch 연동
- 벡터 검색(Vector Search)
- LLM 기반 랭킹 시스템
- Redis 캐시 지원
- 비동기 다운로드 큐
- FastAPI 기반 REST API
- Web UI 제공

개발 방향

이 프로젝트는 다음과 같은 목적에 적합합니다.

개인 검색 엔진

AI 데이터 수집 시스템

논문 검색 플랫폼

미디어 다운로드 시스템

메타데이터 기반 추천 엔진

플러그인 실험 플랫폼

라이선스

MIT License

